

Attack Type	Layer	What attacker needs (requirements)	Key dependency / insight	Typical mitigation
Dictionary / Brute-force attack (online)	Application	- Access to login interface - Ability to send repeated login attempts	Limited by rate limiting and lockout	Rate limiting, CAPTCHA, account lockout
Dictionary / brute-force attack (offline)	Cryptographic	- Captured hash/password file OR ciphertext - Ability to test guesses locally	No interaction with system required	Salted hashes, slow hash functions (bcrypt, Argon2)
WPA2 password cracking (offline)	Link	- Captured WPA2 handshake - Weak PSK (low entropy)	Attack is offline after capture	Strong passphrase, WPA3 SAE
ARP poisoning (MITM setup)	Link	- Same LAN / same broadcast domain - Ability to send ARP replies	ARP is unauthenticated + stateless	Client isolation, DHCP snooping, DAI
DNS spoofing	Application / Network	- MITM position (via ARP or network control) - Ability to intercept DNS query	First-response wins (race condition)	DNSSEC, DoH/DoT, secure resolver
MITM via Wi-Fi (WPA2 break)	Link	- Wi-Fi access (password known or cracked) - Optional: ability to intercept traffic	WPA2 alone does NOT prevent LAN MITM	TLS, certificate validation
Session key recovery (WPA2-PSK)	Link crypto	- Captured 4-way handshake - PSK (password)- Knowledge of derivation process	No forward secrecy in WPA2-PSK	WPA3, ECDHE-based systems
Padding oracle attack (AES-CBC)	Crypto	- Access to decryption oracle OR error feedback - Ability to modify ciphertext/IV	Exploits padding validation leaks	AEAD modes (AES-GCM), disable error leaks
CSRF attack	Web	- Victim authenticated in browser - Ability to trick victim into sending request	Browser auto-attaches cookies	CSRF tokens, SameSite cookies
XSS attack	Web	- Injection point in web app (input reflected) - Browser executes script	Script runs in victim origin	Input sanitisation, CSP, HttpOnly cookies
SQL injection	Web / DB	- Unsanitised user input in SQL query - Ability to inject SQL syntax	Query mixing code + data	Parameterised queries
Password reset poisoning	Web	- Ability to control Host header or URL construction	Server trusts attacker-controlled input	Server-side URL generation
DoS / ICMP flood	Network	- Ability to send large volume of traffic OR spoof IP	Exhausts bandwidth/resources	Rate limiting, firewall rules
Amplification attack (DDoS)	Network	- Access to reflective servers (DNS, NTP, etc.) - Ability to spoof victim IP	Small request → large response ratio	Disable open resolvers
Replay attack (auth protocols)	Protocol	- Captured authentication messages	No freshness guarantee	Nonces, timestamps, challenge-response
Format string attack (printf)	System	- Vulnerable printf-like function - Attacker-controlled format string	Stack leak + memory write primitive	Safe formatting functions
Buffer overflow / stack smashing	System	- Unsafely bounded input buffer - Ability to overwrite memory	Overwrites return address/control flow	ASLR, canaries, bounds checking
Integer overflow exploit	System	- Arithmetic used in memory allocation or bounds check	Wrap-around leads to undersized buffer	Safe integer checks
TOCTTOU attack	OS / Access control	- Race window between check and use - Ability to modify resource state	Time gap vulnerability	Atomic syscalls, fstat checks
Privilege escalation (setuid exploit)	OS	- Misconfigured setuid binary OR vulnerable program	Effective UID change	Principle of least privilege
TLS Renegotiation Attack	Transport (TLS)	- Ability to intercept or sit in MITM position (or inject into TCP stream) - Server/client supporting insecure TLS renegotiation- Ability to trigger or manipulate renegotiation handshake	Exploits TLS renegotiation where new handshake is not properly bound to existing session → attacker can prepend or inject data into authenticated session	Disable insecure renegotiation, use TLS 1.3 (removes renegotiation), bind renegotiation to session keys, use secure renegotiation extension

Topic	Variations	Dependency	Considerations
Encryption and Cryptography	Brute-force / dictionary (offline) Padding oracle (CBC) Stream cipher keystream reuse One-time pad misuse RSA malleability / homomorphism misuse Meet-in-the-middle (2-DES)	Weak randomness, key reuse, missing integrity checks, or oracle leakage	- If attacker does NOT interact with system → offline crypto attack - If you see XOR reuse → stream cipher weakness - If there is error message / padding check → oracle attack - If encryption is “double but still broken” → meet-in-the-middle - If modification possible without detection → no authentication layer (no MAC / AEAD)
Authentication & Identity Attacks	Replay attack Password guessing (online/offline) Phishing / credential theft Password reset poisoning CSRF (session hijacking via browser trust) Biometric spoofing	Lack of freshness, weak identity verification, or trust in user-controlled input	- If attacker reuses old valid message → replay - If browser auto-sends cookies → think CSRF - If user is tricked → phishing/social engineering always default - If login is bypassed via repeated tries → brute force / dictionary - If server trusts headers/user input → auth logic flaw (reset poisoning, CSRF, etc.)
Network	• ARP poisoning (LAN MITM) • DNS spoofing • DoS / DDoS / reflection / amplification • Wi-Fi MITM (LAN compromise) • Packet sniffing / interception	Lack of authentication at network layers + trust in routing/name resolution	- If attack happens within local network → ARP or LAN MITM - If domain name resolved incorrectly → DNS spoofing - If traffic volume explodes → DoS / amplification - If attacker sits between users → always think MITM chain (ARP → DNS → TLS break attempt) - If response is larger than request → amplification clue
Reverse Engineering	• Buffer overflow / stack smashing • Format string attack • Integer overflow • TOCTTOU race condition • Privilege escalation (setuid abuse) • Memory corruption attacks	Unsafe memory handling, lack of bounds checks, race conditions	- If memory is overwritten → think stack/control flow attack - If program leaks memory → format string vulnerability - If value “wraps around” → integer overflow - If attack depends on timing gap → TOCTTOU - If privilege suddenly increases → check effective UID / system calls / SUID binaries
Others	• XSS (reflected / stored) • SQL injection • Cookie/session attacks • TLS downgrade / MITM interception	Improper separation of data vs code + weak input sanitisation + trust in browser/server	- If input becomes executable → XSS or SQL injection - If DB query is modified → SQL injection always first guess - If script runs in browser origin → XSS - If cookies are stolen/used automatically → session-based attack (CSRF/XSS combo) - If HTTPS is mentioned with interception → think TLS/MITM layer failure

Mechanism	Protects Against (Attacker Capability)	CIA Triad	Requirements	Flaws / Breaks
AES (general / block cipher primitive)	COA, CPA (if properly used in secure construction), partial KPA resistance	C + I	Secure key generation, sufficient key length (128/192/256-bit), proper mode of operation	Weak key management, improper mode usage leads to insecurity
ECB (Electronic Code Book)	COA only (very weak security)	C (weak I)	None beyond encryption key (no IV used)	Deterministic: identical plaintext → identical ciphertext, pattern leakage, image/text structure visible
CBC (Cipher Block Chaining)	COA, CPA (if IV is random and unpredictable)	C + I	Random/unpredictable IV, proper padding scheme, secure MAC if integrity needed	Padding oracle attacks, IV reuse weakens security, error propagation, malleability without authentication
CTR (Counter Mode)	COA, CPA (strong if implemented correctly)	C + I	Unique nonce/IV per encryption, never reuse counter stream, secure PRF-based keystream	Keystream reuse is catastrophic (XOR leak), malleability if no MAC, bit-flipping attacks
Stream Cipher	COA, CPA (if IV used properly)	C	PRG security, unique IV	keystream reuse → ($C_1 \oplus C_2 = P_1 \oplus P_2$)
One-Time Pad	COA (perfect secrecy)	C	Truly random key, same length, never reused	Key reuse completely breaks security
RSA Encryption	CPA, passive attacker	C	Hard factoring, correct padding (OAEP), secure key generation	Malleability, deterministic (if unpadded), no forward secrecy
Diffie–Hellman (Ephemeral / Transient Keys)	MITM (if authenticated), passive eavesdropping, key recovery after session	C + I	Large prime ppp, generator ggg, random ephemeral secrets a,ba, ba,b, authentication layer (signatures or certificates), secure nonce use	Vulnerable to MITM if unauthenticated, no identity binding by default, no protection if ephemeral values reused, replay possible if nonces absent
RSA-based Key Exchange (Public Key Encryption of Session Key)	Passive eavesdropping, key interception (if PKI is secure), impersonation (if certificate trusted)	C + I	Valid PKI (certificates), secure RSA key generation, correct padding (e.g. OAEP), secure public key distribution	No perfect forward secrecy (session key recoverable if private key leaked), vulnerable to MITM if fake certificate accepted, malleable if padding weak, heavy computational cost
Hash Functions (SHA-2, SHA-3)	Tampering / data modification	I	Collision resistance, preimage resistance	Collision attacks, length extension (bad design usage)
MAC (HMAC, CBC-MAC)	Forgery, message tampering	C + I	Shared secret key, UF-CMA security, proper construction	No non-repudiation, replay attacks without nonce
Digital Signature	Forgery, impersonation	I + Auth + NR	Private key security, collision-resistant hash	Bad randomness, hash collisions, CA compromise
TLS / SSL	MITM, sniffing, packet modification	C + I	Certificate validation, secure handshake, PFS	Weak cipher suites, CA compromise, downgrade attacks
Password Hashing (bcrypt, Argon2)	Offline dictionary attack, brute-force cracking of stored passwords	C	Salted hashes, slow/iterative hashing (work factor), secure hash function design	No salt → rainbow table attack, weak passwords → fast cracking, low cost factor → GPU/ASIC acceleration breaks
Encrypted Password File (Symmetric Encryption of Password Database)	Database theft (offline access), passive data exposure	C	Strong encryption algorithm (e.g. AES), secure key management, separate key storage from database	Key compromise breaks entire file, does not prevent password guessing if weak passwords used, replay/guessing still possible after decryption
Access Control (DAC, MAC, RBAC)	Unauthorized access, privilege misuse	C + I	Least privilege, reference monitor, correct policy	Privilege escalation, misconfiguration, TOCTTOU
DNSSEC	DNS spoofing, cache poisoning	I	Digital signatures on DNS records	CA/log compromise, operational complexity
WPA2 (Wi-Fi Protected Access II)	Wireless sniffing, passive eavesdropping, unauthorized network access	C + I + (partial A via authentication)	Strong passphrase (PSK) or enterprise authentication (802.1X), AES-CCMP encryption, proper handshake validation	Weak passwords → offline dictionary attack on handshake, does not prevent post-authentication LAN attacks (e.g. ARP poisoning), vulnerable if handshake captured + low entropy PSK
VPN (Virtual Private Network)	Network sniffing, IP-based tracking, MITM on public networks	C + I	Secure tunnel protocol (TLS/IPSec), authenticated endpoint, encrypted traffic encapsulation	Endpoint compromise bypasses VPN, DNS leaks, misconfiguration leaks traffic outside tunnel
IPSec (Internet Protocol Security)	Packet sniffing, IP spoofing, MITM at network layer	C + I	Authentication Header (AH) or ESP, secure key exchange (IKE), encryption + integrity protection at IP layer	Complex configuration, key exchange vulnerabilities, deployment issues, overhead reduces performance
DoS Mitigations (rate limiting, filtering)	Service disruption attacks	A	Traffic filtering, redundancy	Cannot fully prevent large-scale DDoS
ARP protection (static ARP, filtering)	ARP poisoning MITM	C + I	Trusted ARP entries, network control	Manual config errors, local network exposure

CVE — Common Vulnerabilities and Exposures — A standardized public list of known cybersecurity vulnerabilities — Purpose is to provide unique identifiers for vulnerabilities so they can be consistently tracked and referenced across systems and organizations

NIST — National Institute of Standards and Technology — A U.S. government agency that develops technology and cybersecurity standards and guidelines — Purpose is to provide frameworks, best practices, and vulnerability data to improve security and risk management

ISO — International Organization for Standardization — An international body that develops global standards across industries including information security — Purpose is to establish widely accepted standards like ISO 27001 to help organizations implement and certify strong security practices

Quantum computer, affects entropy by square root, need to double bits to get same security

Hash vs Encryption, because hash need to enforce collision resistance need to have double bits to get same security

[illegible]